

Sprint Documentation

Sprint 16	Start Date: 30/04/2026	Final Date: 02/05/2026
Projetc Name:	Civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Gustavo Henrique Moreira Porto		
Giovanni da Silva Nascimento		
Felipe Pacheco Bianchini		

Sprint Backlog

Task#	Description	Start Date	Final date
#91	Adicionar classe Unidade Consumidora	30/04/2026	02/05/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#91	Garantir que o cadastro e manutenção das unidades de medida possuam validações completas de integridade, padronização e regras de negócio, assegurando consistência no uso em tipos de despesa e lançamentos no sistema. Além disso, aplicar correções necessárias no fluxo atual.	Gustavo Henrique Moreira Porto, Giovanni da Silva Nascimento e Felipe Pacheco Bianchini	Completo	2 horas	2 horas

Sprint Description

Tarefa: Sprint 16 - Adicionar classe Unidade Consumidora

Objetivo

Criar a entidade Unidade Consumidora no sistema de gestão de prefeitura, permitindo o cadastro e manutenção das unidades consumidoras vinculadas às instituições, secretarias, tipos de despesa, orçamentos e fornecedores já existentes no sistema.

Além disso, implementar as validações necessárias no backend para garantir integridade dos dados, consistência relacional e regras de negócio.

Descrição

Implementar a classe Unidade Consumidora no backend, contendo os atributos definidos inicialmente e seus respectivos relacionamentos com tabelas já existentes no banco de dados.

A entidade deverá representar uma unidade consumidora utilizada no controle de despesas da prefeitura.

Critérios de Aceite

◆ Criação da Entidade Unidade Consumidora

- Criar a classe UnidadeConsumidora

Criar o mapeamento da entidade no banco de dados

Criar a migration correspondente

Criar a tabela no banco de dados

Garantir que a entidade siga o padrão utilizado no projeto

◆ Atributos da Entidade

A classe deverá conter os seguintes campos:

-
- Id
 - Tipo serial/autoincremento
 - Chave primária da tabela
- Identificador
 - Tipo string
 - Esse campo recebe os dados como RENAVAL, HIDRÔMETRO, MEDIDOR e etc
- IdInstituicao
 - Deve referenciar uma instituição existente no banco
- IdTipoDespesa
 - Deve referenciar um tipo de despesa existente no banco
- IdSecretaria
 - Deve referenciar uma secretaria existente no banco
- IdOrçamento
 - Deve referenciar um orçamento existente no banco
- IdFornecedor
 - Deve referenciar um fornecedor existente no banco
- Excluido
 - Tipo Bool

- Será usado para o soft delete
- DataExclusao
 - Deve ser do tipo Date (analisar qual usar)
 - Deve ser null por padrão e deve ser preenchido quando o registro for excluído e retornado a null se for o registro for restaurado.

◆ Validações de Campos Obrigatórios

- Validar que os campos obrigatórios não estão vazios:
 - Identificador
 - IdInstituicao
 - IdTipoDespesa
 - IdSecretaria
 - IdOrcamento
 - IdFornecedor

◆ Validação de Identificador

- O campo deve ser criado como string e permitir combinação de caracteres diferentes.

◆ Validação de Instituição

- idInstituicao deve existir no banco

Validar integridade da chave estrangeira

Garantir que a instituição esteja válida para uso

◆ Validação de Tipo de Despesa

- idTipoDespesa deve existir no banco

Validar integridade da chave estrangeira

Garantir que o tipo de despesa esteja ativo, caso exista controle de situação

Não permitir vínculo com tipo de despesa inexistente ou inválido

◆ Validação de Secretaria

- idSecretaria deve existir no banco

Validar integridade da chave estrangeira

Garantir que a secretaria esteja válida para uso

◆ Validação de Orçamento

- idOrçamento deve existir no banco

Validar integridade da chave estrangeira

Garantir que o orçamento informado esteja válido para uso

◆ Validação de Fornecedor

- idFornecedor deve existir no banco

Validar integridade da chave estrangeira

Garantir que o fornecedor esteja válido para uso

◆ Validação de Excluído

- Por padrão, o valor false deve ser setado em novos registros

◆ Relacionamentos

- Criar os relacionamentos da entidade UnidadeConsumidora com:
 - Instituicao
 - TipoDespesa
 - Secretaria
 - Orcamento
 - Fornecedor

Garantir que as chaves estrangeiras estejam configuradas corretamente

Verificar se os nomes dos relacionamentos seguem o padrão do projeto

◆ Regras de Negócio

- Não permitir cadastro de unidade consumidora com referências inexistentes

Não permitir duplicidade de unidade consumidora com o mesmo identificador

Garantir que apenas registros válidos possam ser utilizados

◆ Validação em Atualização

- Ao atualizar uma unidade consumidora:
 - Validar novamente todos os relacionamentos
 - Garantir que os novos IDs informados existam no banco
 - Manter a consistência dos dados após alteração

◆ Validação de Exclusão - Soft Delete

- Caso existam registros vinculados, a exclusão deve ser bloqueada e retornar erro

Impedir o uso de unidades consumidoras excluídas em novos lançamentos

◆ Validação de DataExclusao

- O campo deve ser null em novos registros

Quando registro for excluído o campo deve receber a data de exclusão. Quando for restaurado, a data deve retornar a null

◆ Normalização e Boas Práticas

- Implementar validações na camada SERVICE

Não depender apenas de DataAnnotations

Validar dados antes de persistir no banco

Retornar mensagens claras ao usuário

Os erros retornados devem vir em uma lista com todos os erros encontrados de uma só vez

Seguir o padrão de arquitetura já existente no projeto

Verificar o projeto antes do envio

Endpoints e Resultado Esperado

Aqui estão os endpoints que esperamos ver na entrega da tarefa:

GET /unidades-consumidoras - Retorna apenas os dados que não foram excluídos pelo Soft Delete.

GET /unidades-consumidoras/excluidos - Retorna apenas os dados que foram excluídos pelo Soft Delete

GET /unidades-consumidoras/{id} - Retornar apenas os que não estão excluídos

GET /unidades-consumidoras/identificador/{identificador} - Retornar apenas os que não estão excluídos

GET /unidades-consumidoras/instituicao/{idInstituicao} - Retornar apenas os que não estão excluídos

GET /unidades-consumidoras/secretaria/{idSecretaria} - Retornar apenas os que não estão excluídos

POST /unidades-consumidoras - Cadastrar novas unidades consumidoras

PUT /unidades-consumidoras/{id} - Editar unidades consumidoras

PATCH /unidades-consumidoras/{id}/status-exclusao - Esse aqui é um softdelete. Para funcionar, adicione o atributo public bool Excluido { get; set; } na classe e trate como: False = Não foi Excluido; True = Foi Excluido. Por padrão, todos os novos registros devem ser setados como false. O registro "excluído" NÃO DEVE ser removido do banco.

Observações e Sugestões

- A entidade Unidade Consumidora deve ser criada pensando em futuras expansões.
- Caso novos campos sejam necessários, eles deverão ser adicionados posteriormente.
- Garantir que os vínculos com instituição, secretaria, orçamento, fornecedor e tipo de despesa estejam consistentes.
- Evitar cadastros duplicados com a mesma combinação de dados.
- Manter as validações centralizadas para facilitar manutenção futura.

- Verifiquem o projeto antes do envio.

Sprint Results

Foi criado:

- Criação da entidade UnidadeConsumidora com 5 chaves estrangeiras, soft delete (Excluido + DataExclusao) e navigation properties.
- Mapeamento da entidade no banco via UnidadeConsumidoraBuilder (Fluent API): chave primária, Identificador obrigatório com MaxLength(100) e índice único, FKs com OnDelete(DeleteBehavior.Restrict), Excluido com HasDefaultValue(false), DataExclusao opcional.
- DTO UnidadeConsumidoraDTO espelhando o modelo, com mapeamento bidirecional registrado em MappingsProfile.
- Repositório UnidadeConsumidoraRepository herdando GenericRepository e expondo: paginação por status, busca por id/identificador filtrando excluídos, busca por instituição e secretaria, checagem de unicidade do identificador (com ignoreId) e validadores de existência das 5 FKs.
- Service UnidadeConsumidoraService herdando GenericService, com override de Create e Update aplicando o pipeline ValidateDtoInstance → Normalize → ValidateFields → ValidateRelationships → ValidateUniqueness, agregação de erros em List<string>, e método ToggleStatusExclusaoAsync para soft delete reversível.
- Exceções UnidadeConsumidoraValidationException (lista de erros) e UnidadeConsumidoraConflictException (conflito com Field), seguindo o padrão de InstituicaoValidationException / InstituicaoConflictException.
- Controller UnidadeConsumidoraController com 9 endpoints, [Authorize], tratamento padronizado de exceções e envelope Response.
- Registro em Program.cs (DI) e em ApplicationDbContext (DbSet + builder).
- Migration AddUnidadeConsumidora gerada via dotnet ef migrations add (snapshot atualizado e versionado; arquivo de migration permanece local conforme política do .gitignore).

Diagrama de Casos de Uso:

Administrador

- Cadastrar Unidade Consumidora — include: Validação de UC (obrigatórios + FKs + unicidade)
- Atualizar Unidade Consumidora — include: Validação de UC + bloqueio de edição em excluídos
- Listar Unidades Consumidoras — include: Filtro de Soft Delete (Excluido = false)
- Buscar por Identificador / Instituição / Secretaria
- Excluir Logicamente (soft delete) — include: Soft Delete (Excluido = true; DataExclusao =.UtcNow)
- Restaurar Unidade Consumidora — include: Soft Delete (Excluido = false; DataExclusao = null)

Diagrama de Classes Afetadas:

UnidadeConsumidora

- int Id
- string Identificador
- int IdInstituicao, int IdTipoDespesa, int IdSecretaria, int IdOrcamento, int IdFornecedor
- bool Excluido
- DateTime? DataExclusao
- Navigation: Instituicao, TipoDespesa, Secretaria, Orcamento, Fornecedor

UnidadeConsumidoraDTO (mapeado via AutoMapper)

- Mesmas propriedades da entidade, sem navigation properties.

IUnidadeConsumidoraRepository (estende IGenericRepository)

- GetPageNaoExcluidos(PaginationQuery)
- GetPageExcluidos(PaginationQuery)
- GetByIdNaoExcluidoAsync(int)
- GetByIdIdentificadorNaoExcluidoAsync(string)
- GetByIdInstituicaoNaoExcluidosAsync(int)
- GetByIdSecretariaNaoExcluidosAsync(int)
- ExistsByIdentificadorAsync(string, int? ignoreId)
- InstituicaoExistsAsync(int), TipoDespesaExistsAsync(int), SecretariaExistsAsync(int), OrcamentoExistsAsync(int), FornecedorExistsAsync(int)

UnidadeConsumidoraService (estende GenericService)

- Create(dto), Update(dto, id) com override
- ToggleStatusExclusaoAsync(id)
- Privados: Normalize, ValidateFields, ValidateRelationships, ValidateUniqueness

UnidadeConsumidoraController

- Os 9 endpoints da spec.

Exceções

- UnidadeConsumidoraValidationException (lista IReadOnlyCollection<string> Errors)
- UnidadeConsumidoraConflictException (campo string? Field)

Diagrama de Sequência — Cadastro com Validação Agregada:

1. Administrador envia POST /api/unidades-consumidoras com payload.
2. Controller chama Service.Create(dto).
3. Service executa ValidateDtoInstance(dto) → Normalize(dto) → ValidateFields(dto).

4. Se houver erros nos campos, lança `UnidadeConsumidoraValidationException` com a lista de erros, controller retorna 400 BadRequest com Data populado.
5. Se campos válidos, executa `ValidateRelationships(dto)` checando cada FK no banco. Se inválida, lança `ValidationException` agregando erros.
6. Executa `ValidateUniqueness(dto)`. Se identificador já cadastrado, lança `UnidadeConsumidoraConflictException` (Field = "identificador"), controller retorna 409 Conflict.
7. Se tudo válido, força `dto.Excluido = false` e `dto.DataExclusao = null`, persiste via `Repository.Add(entity)`.
8. Controller retorna 200 OK com o DTO contendo o Id gerado pelo banco.

Diagrama de Sequência — Soft Delete via Toggle:

1. Administrador envia PATCH `/api/unidades-consumidoras/{id}/status-exclusao`.
2. Controller chama `Service.ToggleStatusExclusaoAsync(id)`.
3. Service busca o registro com `Repository.GetById(id)`. Se inexistente, lança `KeyNotFoundException`, controller retorna 404 NotFound.
4. Se `existing.Excluido == false`: define `Excluido = true` e `DataExclusao = DateTime.UtcNow`.
5. Se `existing.Excluido == true`: define `Excluido = false` e `DataExclusao = null`.
6. Persiste via `Repository.Update(existing)`.
7. Controller retorna 200 OK com { Id, Excluido, DataExclusao }.

Testes realizados

Verificações de build e migração:

- `dotnet build Civitas.WebAPI/Civitas.WebAPI.csproj` — 0 erros, 92 avisos (todos pré-existent, CS8618 nullable do código legado).
- `dotnet ef migrations add AddUnidadeConsumidora` — migration gerada com sucesso, snapshot atualizado e revisado (cria a tabela `unidadeconsumidora`, configura as 5 FKs com Restrict, índice único em identificador, default de `excluido=false`, `dataexclusao nullable`, índices nas 5 FKs).

Testes manuais (Swagger UI):

- POST com Identificador em branco e FKs zeradas → 400 com lista contendo "O campo Identificador e obrigatorio.", "O campo Instituicao e obrigatorio." e demais.
- POST com FKs apontando para registros inexistentes → 400 com mensagens "A instituicao informada nao foi encontrada." etc., todas em uma única lista.
- POST com Identificador igual a um já cadastrado e ativo → 409 com Field = "identificador".
- POST válido → 200 com Id preenchido pelo banco, Excluido = false, DataExclusao = null.
- PUT em UC com Excluido = true → 400 com mensagem orientando a restauração antes de editar.
- PUT alterando Identificador para um valor já em uso por outra UC → 409.
- PATCH `/status-exclusao` em UC ativa → 200, registro fica Excluido = true, DataExclusao preenchido com timestamp UTC.

- PATCH /status-exclusao em UC excluída → 200, registro retorna Excluido = false, DataExclusao = null.
- GET /api/unidades-consumidoras → retorna apenas registros com Excluido = false.
- GET /api/unidades-consumidoras/excluidos → retorna apenas registros com Excluido = true.
- GET por id, identificador, instituição e secretaria → todos filtram Excluido = false.

Assinaturas

Documento assinado digitalmente
 **GUSTAVO HENRIQUE MOREIRA PORTO**
Data: 01/05/2026 12:28:12-0300
Verifique em <https://validar.iti.gov.br>

Documento assinado digitalmente
 **FELIPE PACHECO BIANCHINI**
Data: 01/05/2026 12:33:26-0300
Verifique em <https://validar.iti.gov.br>

Documento assinado digitalmente
 **GIOVANNI DA SILVA NASCIMENTO**
Data: 01/05/2026 12:35:51-0300
Verifique em <https://validar.iti.gov.br>